electronics

MDPI

Article

Structure-Aware Lightweight Document-Level Event Extraction via Code-Based Large Language Models

Xing Xu [1], Jianbin Zhao [1], Pengfei Zhang [1], Yaduo Liu [1], Bingyang Yu [1], Puyuan Zheng [2], Dingyuan Hu [2], Zhongchen Deng [2], Ping Zong [2], Guoxin Zhang [2], Zhonghong Ou [3,*], Meina Song [2] and Yifan Zhu [2]

[1] State Grid Hebei Information and Telecommunication Branch, Shijiazhuang 050011, China; hsuxing@zju.edu.cn (X.X.); lingd2026@163.com (J.Z.); zhangpengfei9201@126.com (P.Z.); yaduoliu@163.com (Y.L.); ronan85713@163.com (B.Y.)

[2] School of Computer Science (National Pilot Software Engineering School), Beijing University of Posts and Telecommunications, Beijing 100876, China; 2025010335@bupt.cn (P.Z.); fcdwhcdy@126.com (D.H.); dzckepwalk@gmail.com (Z.D.); zongping8912@bupt.edu.cn (P.Z.); guoxin.zhang@bupt.edu.cn (G.Z.); mnsong@bupt.edu.cn (M.S.); yifan_zhu@bupt.edu.cn (Y.Z.)

[3] State Key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing 100876, China

* Correspondence: zhonghong.ou@bupt.edu.cn

Abstract

Document-level Event Extraction (DEE) requires identifying complex event records and arguments dispersed across unstructured texts. However, applying general Large Language Models (LLMs) to DEE is intrinsically hindered by their lack of inductive bias for rigid structural constraints, often leading to schema violations and suboptimal performance in complex structural prediction tasks. To address this, we propose the S tructure-Aware Lightweight DEE, termed **SALE**, which leverages the structural reasoning potential of Code-Based LLMs (Code-LLMs) as a favorable inductive preference. We leverage the natural isomorphism between event schemas and programming object definitions, formulating event extraction as a Python 3.9 class instantiation task to bridge the gap between semantic understanding and structural adherence. Specifically, SALE employs a novel two-stage training paradigm: First, a Structure-Aware Fine-tuning stage injects general structural knowledge via diverse code-style instruction tasks derived from broad Information Extraction (IE) datasets; second, an Event Extraction Alignment stage utilizes a reward-based alignment loss—optimized via policy gradient—to adapt this capability to document-level intricacies. The effectiveness of SALE stems from the synergy between its structure-aware prompting and the specialized alignment stage built on a code-oriented backbone. Extensive experiments on established news-domain benchmarks (RAMS and WikiEvents) demonstrate that our approach significantly outperforms representative supervised and general LLM baselines in cross-task zero-shot and few-shot transfer settings (e.g., surpassing supervised baselines by over 7% in F1 score). Furthermore, SALE maintains a highly efficient inference profile and parameter-efficient footprint, offering a practical and scalable solution for vertical domain applications.

Keywords: event extraction; code-based Large Language Models; instruction tuning; structure-aware fine-tuning; parameter-efficient fine-tuning code-based Large Language Models; instruction tuning; structure-aware fine-tuning; parameter-efficient fine-tuning

Academic Editor: Arkaitz Zubiaga

Received: 4 February 2026
Revised: 6 March 2026
Accepted: 8 March 2026
Published: 12 March 2026

1. Introduction

Information Extraction (IE), particularly Event Extraction (EE) [1,2], serves as a pivotal technology in Natural Language Processing (NLP) [3], facilitating the transformation of unstructured narratives into structured knowledge graphs for downstream applications such as intelligent question answering and decision support systems. While sentence-level extraction has been widely studied, real-world scenarios—ranging from financial risk analysis to legal document processing—often necessitate Document-level Event Extraction (DEE). Unlike its sentence-level counterpart, DEE requires identifying event triggers and arguments dispersed across multiple sentences, intrinsically demanding a model capable of long-range dependency modeling and complex context understanding.

However, deploying effective generative DEE systems faces a fundamental challenge: the mismatch of inductive bias. Standard Large Language Models (LLMs) are optimized for the flexible generation of natural language, lacking the intrinsic sensitivity to the rigid schema constraints required by structured prediction tasks. Consequently, when applied to complex DEE scenarios involving cross-sentence relationships and multi-event coexistence [4–6], general LLMs frequently exhibit "structural hallucinations"—generating semantically plausible but structurally invalid outputs. While scaling up to giant models (e.g., GPT-4) can mitigate this via emergent capabilities, the resulting computational overhead and privacy concerns render them impractical for many vertical domain applications. Thus, a critical question arises: How can we achieve precise, structure-aware event extraction with lightweight models?

To answer this, we turn to Code-Based LLMs (Code-LLMs). We posit a natural isomorphism between event extraction schemas and programming object definitions: Both require strict adherence to syntax, type constraints, and hierarchical logic. Unlike general LLMs, Code-LLMs exhibit a promising structural sensitivity acquired through pre-training on rigorous programming languages [7]. While the pre-training provides a practical foundation for structured generation, this structural sensitivity offers a promising yet underexplored avenue to bridge the gap between semantic understanding and schema adherence when combined with targeted instruction-tuning and alignment.

In this paper, we propose **SALE** (Structure-Aware Lightweight Document-level Event Extraction), a novel framework that reformulates event extraction as a Python class instantiation task. By leveraging the code-centric capabilities of lightweight Code-LLMs, SALE effectively navigates the complexities of DEE. Our methodology employs a progressive two-stage training paradigm. The first stage, Structure-Aware Fine-tuning, injects general structural knowledge into the model by utilizing diverse IE datasets transformed into Python-style instruction tasks. The second stage, Event Extraction Alignment, adapts this capability to the specific intricacies of DEE, employing a reinforcement learning-inspired alignment objective to optimize cross-sentence reasoning and argument identification.

The main contributions of this paper are summarized as follows:

- We introduce a novel perspective that bridges the gap between Information Extraction and Code Generation. By treating event extraction as a Python class instantiation process, we propose a comprehensive paradigm combining Python-style structured prompting and a two-stage adaptation strategy built on a code-oriented backbone. This framework empirically improves schema adherence and structured extraction, thereby minimizing schema violations.
- We propose SALE, a structure-aware lightweight framework that integrates a two-stage training strategy—comprising general structure injection and task-specific alignment—to effectively handle long-range dependencies and multi-event scenarios using only 7B parameters.

https://doi.org/10.3390/electronics15061187

- We conduct extensive experiments on the RAMS and WikiEvents benchmarks. The results demonstrate that SALE significantly outperforms state-of-the-art baselines, particularly in low-resource settings (cross-task zero-shot and few-shot transfer), achieving substantial improvements in Argument Identification and Classification metrics.

2. Related Work

2.1. Document-Level Event Extraction

Early work on event extraction predominantly focused on sentence-level settings, which inherently struggle to capture arguments dispersed across sentences. To provide a clear taxonomy of the current research landscape and identify the existing gaps, we summarize representative Document-level Event Extraction (DEE) and generative IE methods in Table 1. As shown in the first five rows, recent studies have explicitly pivoted to Document-level Event Argument Extraction (DEAE) to address real-world complexities. Li et al. [8] formulated DEAE as conditional generation over event templates, demonstrating that modeling cross-sentence arguments improves the coverage of participants. To better handle long contexts, Peng et al. [9] proposed a cooperative framework to separate event boundary detection from argument extraction, while Yu et al. [10] designed dual-decoder architectures to jointly model arguments and roles. Furthermore, Zhou et al. [11] and Zhang et al. [12] introduced argument constraint enhancement and AMR-based sparse representations, respectively, to filter noisy contexts. Recent research [13] also highlights the challenges of multi-document integration. Despite these advances, a significant gap remains: As highlighted in the "Disadvantage" column of Table 1, most supervised methods rely on complex, task-specific architectures that are difficult to generalize and often resource intensive.

Table 1. Summary of Existing DEE, Generative and Instruction-Driven IE Methods.

Reference	Method	Disadvantage	Performance
Li et al. [8]	Formulates DEAE as conditional generation over event templates	Rely on complex and task-specific architectures	Are difficult to generalize and often resource-intensive
Peng et al. [9]	Employs a cooperative framework		
Yu et al. [10]	Designs a dual-decoder architecture		
Zhou et al. [11]	Introduces argument constraint enhancement mechanisms		
Zhang et al. [12]	Utilizes AMR-based sparse representations		
Wang et al. [14]	Abstracts IE into universal relations	Lack an intrinsic inductive bias for rigid output schemas	Often lead to "structural hallucinations"
Wang et al. [15], Jiao et al. [16]	Fine-tune LLMs		
Hsu et al. [17], Lin et al. [18]	Employ schema-based prompts		

2.2. Generative and Instruction-Driven IE

The paradigm of Information Extraction has shifted significantly from sequence labeling to conditional generation. Approaches like QA-based extraction [19,20] and unified frameworks like UIE [21] and LasUIE [22] cast diverse IE tasks into a unified sequence-to-sequence format. The last three rows of Table 1 summarize the recent methods for generative and instruction-driven IE. TRUE-UIE [14] further abstracts IE into universal relations to facilitate transfer learning. With the rise of Large Language Models, instruction tuning has been applied to IE. InstructUIE [15] and other works [16] fine-tune LLMs to follow natural language instructions for extracting structured information. Prompt-driven methods like DEGREE [17] and GEMS [18] employ schema-based prompts to guide generation. However, a critical limitation remains: These approaches predominantly utilize General LLMs trained on natural language. As detailed in the performance gaps of Table 1 and noted in recent analyses [1,2], general LLMs lack an intrinsic inductive bias for rigid output schemas. This deficiency frequently results in "structural hallucinations" where the generated output violates the required format, especially in complex DEE tasks. This motivates our shift toward Code-based LLMs to leverage their native structural priors.

2.3. Code-LLMs for Structural Prediction

Code-Based LLMs have exhibited remarkable capabilities in structural reasoning, extending well beyond software engineering [7]. The pre-training objective of code generation inherently demands strict adherence to syntax, type constraints, and hierarchical logic, fostering a unique inductive bias for structured data. While Code-LLMs have been explored for reasoning chains, their application in Document-level Event Extraction remains underexplored. Our work identifies a natural isomorphism between event schemas and programming class definitions. Unlike prior works that struggle to force General LLMs to obey schemas, we explicitly leverage the native structural alignment of Code-LLMs to bridge the gap between semantic understanding and structural adherence.

2.4. Parameter-Efficient Fine-Tuning

To achieve lightweight adaptation, we leverage Parameter-efficient Fine-tuning (PEFT), which freezes the backbone and updates only a small set of parameters. Techniques like Prefix-Tuning [23] and P-Tuning v2 [24] optimize continuous vectors to condition generation. Adapter-based methods [25] insert small modules, while prompt tuning [26] has scaled to billion-parameter models. Compared with full fine-tuning, PEFT drastically reduces memory overhead and is naturally compatible with instruction-style inputs. These properties make it particularly attractive for our **SALE** framework, where we align a structure-aware Code-LLM backbone with document-level event extraction via Python-style templates, ensuring high performance with a minimal parameter footprint.

Recently, constrained decoding methods and JSON schema applications have been proposed to ensure structural validity by manipulating output logits during inference. While effective in guaranteeing valid syntax, these inference-time interventions do not inherently enhance the model's semantic reasoning for complex cross-sentence dependencies. In contrast, SALE operates at the representation and alignment stage. By reformulating extraction as Python class instantiation, SALE leverages the intrinsic code-inductive bias of Code-LLMs to deeply improve structural reasoning (e.g., mitigating role confusion), making it orthogonal and complementary to restricted decoding techniques.

3. Methodology

In this section, we present **SALE** (Structure-Aware Lightweight Event extraction), a framework designed to bridge the gap between the generative capabilities of LLMs

and the rigid schema requirements of Document-level Event Extraction (DEE). We strictly formulate DEE as a Python class instantiation task. Section 3.1 provides a holistic overview of the training paradigm. Section 3.2 details the Structure-Aware Fine-tuning stage, which instills general structural inductive bias using code-style prompts. Finally, Section 3.3 describes the Event Extraction Alignment stage, where we align this capability to complex document-level contexts.

3.1. Overview

To address the "mismatch of inductive bias" in general LLMs—specifically their struggle with cross-sentence dependencies and multi-event coexistence—SALE leverage the inherent structural reasoning of Code-LLMs. As illustrated in Figure 1, our methodology unfolds in two progressive stages:

Stage 1: Structure-Aware Fine-tuning. The primary objective is to activate the Code-LLM's latent structural knowledge for Information Extraction tasks. We construct a unified instruction-tuning corpus from diverse IE datasets (NER, RE, EE) [7]. By casting these heterogeneous tasks into a standardized Python class format, we inject fundamental structural constraints into the model, enabling it to recognize entities and relations as "code objects" rather than mere text spans.

Stage 2: Event Extraction Alignment. While Stage 1 ensures structural validity, the model may still struggle with the specific intricacies of DEE, such as identifying arguments scattered across long documents. This stage aligns the structure-aware backbone specifically to DEE tasks (e.g., RAMS, WikiEvents). We employ a reinforcement-learning-inspired alignment objective to optimize the extraction of triggers and arguments, effectively mitigating hallucinations in long-context reasoning.

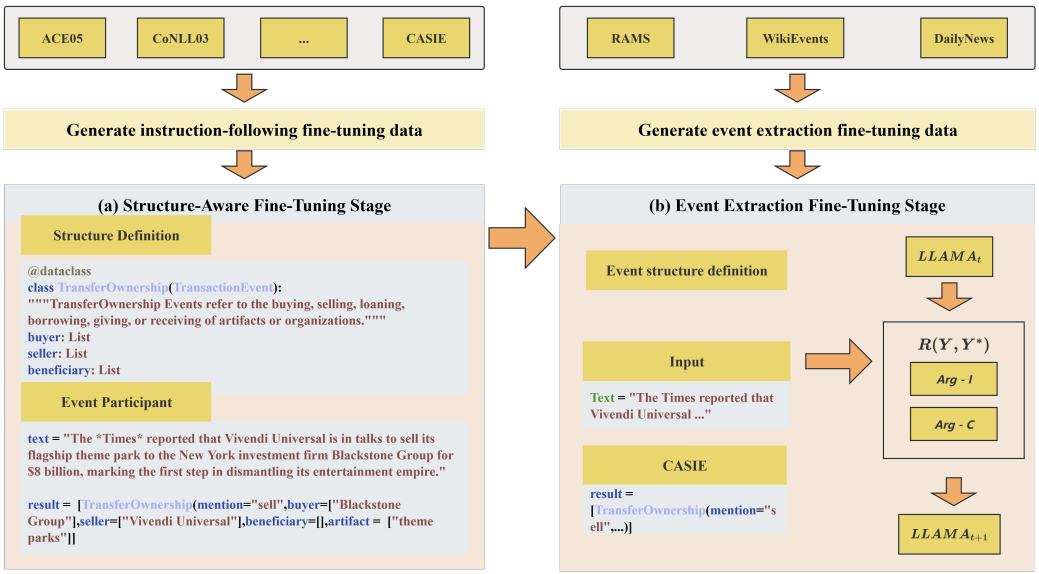

Figure 1. Schematic overview of the proposed SALE. The training paradigm consists of two progressive stages aimed at bridging the gap between semantic understanding and structural adherence. (**a**) Stage 1: Structure-Aware Fine-tuning. We construct a unified instruction-tuning corpus by aggregating diverse Information Extraction (IE) datasets (e.g., NER, RE). These are transformed into Python class-style prompts, where event extraction is formulated as a class instantiation task. This stage injects general structural inductive bias into the Code-LLM backbone. (**b**) Stage 2: Event Extraction Alignment. The model is adapted specifically for Document-level Event Extraction (DEE) tasks (e.g., RAMS, WikiEvents). A specialized alignment objective (rewarding Arg-I and Arg-C) is employed to optimize cross-sentence reasoning and handle multi-event coexistence, utilizing Low-Rank Adaptation (LoRA) to maintain parameter efficiency.

3.2. Structure-Aware Fine-Tuning

Traditional event extraction approaches often decouple NER and argument extraction into isolated sub-tasks, a strategy that frequently fragments the global semantic context and hinders the model's ability to capture long-range dependencies. To overcome this limitation, we propose a Structure-Aware Fine-tuning approach that aggregates a diverse array of IE datasets to train a unified model, which is capable for understanding and reconstructing complex semantic structures from unstructured text.

3.2.1. Unified Information Extraction Datasets

To build a robust base model, we aggregate widely recognized IE benchmarks, including ACE05, CoNLL-2003, and BC5CDR. As detailed in Table 2, this heterogeneous corpus covers diverse domains (News, Biomedical) and tasks (NER, RE, EE). The inclusion of diverse datasets, such as the biomedical-focused BC5CDR and the science-oriented FabNER, ensures that the model is exposed to varied semantic structures from the outset. This cross-domain variety allows SALE to acquire a preliminary capability for structural transferability, laying a foundation for generalized Information Extraction (IE) across heterogeneous vertical domains prior to the task-specific alignment stage. Training on this unified data allows the model to learn a generalized representation of "structure," where identifying a *Person* entity or a *Disease* entity is treated as identifying valid arguments for specific class constructors.

Table 2. Statistics of the Unified Information Extraction Fine-tuning Datasets. The checkmark (✓) indicates that the dataset supports the corresponding task, while the hyphen (-) indicates it does not.

Dataset	Domain	NER	RE	EE	EAE	Count
ACE05	News	✓	✓	✓	✓	19,217
BC5CDR	Biomedical	✓	✓	-	-	4561
CoNLL 2003	News	✓	-	-	-	14,041
DIANN	Biomedical	✓	-	-	-	3976
Ontonotes 5	News	✓	-	-	-	30,000
TACRED	News	✓	✓	-	-	10,027
WNUT 2017	News	✓	-	-	-	3394
BroadTwitter	Twitter	✓	-	-	-	2002
FabNER	Science	✓	-	-	-	2064
Total						**95,566**

3.2.2. Structural Isomorphism via Python Prompts

A core innovation of SALE is utilizing the natural isomorphism between event schemas and programming object definitions. To provide a rigorous theoretical grounding, we formalize this isomorphism as a schema-induced typed structured prediction task. Let $\mathcal{S} = (\mathcal{T}, \mathcal{R})$ represent the event schema, where $\mathcal{T}$ is the set of event types and $\mathcal{R}$ is the set of associated argument roles. We define an abstraction mapping $\Phi : \mathcal{S} \to \mathcal{P}$, which transforms the semantic schema $\mathcal{S}$ into a programmatic class space $\mathcal{P}$, where each $t \in \mathcal{T}$ is represented as a typed class definition with attributes corresponding to roles $r \in \mathcal{R}$.

Given an input document X, the goal of DEE is to identify a set of event instances y^*. We formulate the extraction process as finding the optimal instantiation $y^* \in \mathcal{O}_\mathcal{P}$ that maximizes the conditional probability:

$$y^* = \arg\max_{y \in \mathcal{O}_\mathcal{P}} P(y|X, \Phi(\mathcal{S})) \tag{1}$$

where $\mathcal{O}_\mathcal{P}$ denotes the space of valid object instances strictly constrained by the syntax and type definitions in $\mathcal{P}$. The output is generated via a serialization mapping $\sigma(y)$, which

produces a code-style string (e.g., Python class instantiation). By casting DEE into this programmatic framework, we implicitly leverage the structural constraints of $\mathcal{P}$ to guide the generation process, ensuring that the predicted arguments and triggers satisfy the predefined schema requirements.

We design Python class-style prompts to encapsulate extraction tasks, transforming the ambiguous natural language objective into a rigorous code completion problem, as illustrated in Figure 2. Our prompt template consists of three integral components:

Schema Definition (The Class): We define the target event structure using a Python `@dataclass`. For instance, a `TransferOwnership` event is defined as a class inheriting from a base `Event`. This explicitly communicates the expected output schema to the Code-LLM in its native language.

Semantic Constraints (The Docstrings): Within the class, we embed detailed comments and type hints (e.g., `buyer: List[str]`). These act as semantic guidelines, helping the model map complex narrative roles to specific class attributes.

Instantiation Target (The Code Completion): The input document is presented as a string variable (e.g., `text = ‘‘...’’`), and the model is instructed to instantiate the defined class (e.g., `result = [TransferOwnership(...)]`). By converting DEE into a code instantiation task, we leverage the Code-LLM's pre-trained capability to handle syntax and logical consistency, significantly reducing structural errors.

```
from dataclasses import dataclass
from typing import List

@dataclass
class TransferOwnership(TransactionEvent):
"""TransferOwnership Events refer to the buying, selling, loaning, borrowing,
giving ,or reiceiving of artifacts or organizations."""

buyer: List #The buying agent
seller: List #The selling agent
beneficiary: List #The agent that benefits from the transaction
artifact: List #The item or Organization that was bought or sold
price: List #The sale price takes place
place: List #Where the sale takes place

#unprocessed text
text = "According to the report, Tencent group will sell its subsidiary to a
Shanghai based investment company for 500billion RMB to optimize its business
layout."

#extracted event instances
result = [
TransferOwnership(
  mention = "sell",
  buyer = ["Shanghai investment company"],
  beneficiary = [],
  artifact = ["subsidiary"],
  price = ["500billion RMB"],
  time = [],
  place = ["Shanghai"],
),
]
```

Figure 2. An illustrative example of the Python class-style prompt used in SALE. The prompt defines the event schema (e.g., `TransferOwnership`) using a Python `@dataclass` with detailed type hints and docstrings. The model is tasked with instantiating this class based on the input text, effectively converting event extraction into a code instantiation task.

3.2.3. Base Model Selection

We select CodeLLaMa-7B as our backbone. As shown in our preliminary analysis (Table 3), CodeLLaMa-7B shows a modest but consistent competitive edge over its general text counterpart (LLaMa2-7B) on structural extraction tasks (Avg. 73.3% vs. 73.0%). While the overall average difference is modest, CodeLLaMa-7B demonstrates a more distinct advantage on tasks requiring complex structural generation, such as Event Extraction

(ACE05 EE) and Relation Extraction (TACRED). This finding aligns with our hypothesis that code-pretraining provides a superior inductive bias for complex structural IE.

It is important to clarify that while the scale of the unified instruction-tuning corpus (over 95,000 instances) provides a rich knowledge base, the performance gains are primarily driven by the code-centric paradigm rather than data volume alone. As demonstrated in Table 3, when both models are trained on the identical large-scale corpus, the code-oriented backbone consistently maintains a performance margin over the general-text model in complex structural tasks. Although the average F1 difference of 0.3% is relatively small, these results suggest that code-based pre-training serves as a favorable inductive preference for program-style generation. We emphasize that this pre-training alone is not a stand-alone causal explanation for the final performance; instead, the observed gains in SALE reflect the synergistic effect of the code-oriented backbone, our schema-aware structured prompting, and the proposed two-stage training strategy.

Table 3. Performance Comparison of Base Models on IE Tasks (F1 Score).

Dataset (Task)	CodeLLaMa-7B	LLaMa2-7B
ACE05 (NER)	88.1	**89.1**
ACE05 (RE)	63.6	63.8
ACE05 (EE)	**72.2**	71.7
BC5CDR	87.5	87.5
CoNLL 2003	92.8	92.9
DIANN	79.4	80.3
NCBIDisease	85.4	86.2
Ontonotes 5	83.4	83.4
TACRED	**57.1**	56.6
WNUT 2017	52.0	**53.7**
Average	**73.3**	73.0

This fine-tuning process effectively transforms a general-purpose language model into a specialized Information Extraction model, significantly enhancing its cross-task zero-shot and few-shot transfer performance on downstream event extraction tasks.

3.3. Event Extraction Alignment

The second stage focuses on adapting the structure-aware model to the specific challenges of Document-level Event Extraction, particularly long-range dependency modeling and structural consistency. We construct instruction-tuning data from DEE benchmarks (RAMS, WikiEvents) using the same Python templates described above.

To formalize this process, let an event schema be defined as $S = (T, R, \phi)$, where T is the set of event types, R is the set of argument roles, and $\phi : T \to 2^R$ maps each event type $t \in T$ to its admissible role set $\phi(t)$. Given a document x, the target output is a set of event instances $Y = \{e_i\}_{i=1}^{m}$, where each instance is $e_i = (t_i, \tau_i, A_i)$. Here, τ_i is the trigger span and $A_i = \{(r, a_r) | r \in \phi(t_i)\}$ is the role-argument assignment set. We define a schema-conditioned serialization mapping $g_s : Y \to P_s$ to convert the structured output Y into a canonical class-instantiation representation P_s (Python syntax). Conversely, a deterministic parsing function $h_s : P_s \to Y$ ensures the mapping is information-preserving, i.e., $h_s(g_s(Y)) = Y$.

To bridge the gap between the likelihood-based training objective and the evaluation metrics (F1-score), while further mitigating structural hallucinations in complex contexts, we introduce a specialized alignment objective based on the REINFORCE algorithm [27]. Specifically, given an input text X, the model (acting as a policy π_θ) generates a predicted event structure sequence Y by sampling from its output distribution. To optimize the model

parameters θ directly against the task performance, we define a reward function $R(Y, Y^*)$ that evaluates the quality of the generation against the gold label Y^*, focusing on Argument Identification (Arg-I) and Classification (Arg-C):

$$R(Y, Y^*) = \text{Arg-I}(Y, Y^*) \times \text{Arg-C}(Y, Y^*) \times \text{ArgumentF1}(Y, Y^*). \quad (2)$$

The training objective is to maximize the expected reward $J(\theta) = \mathbb{E}_{Y \sim \pi_\theta(\cdot|X)}[R(Y, Y^*)]$. To reduce the variance of the gradient estimation during the reinforcement learning process, we introduce a baseline b, calculated as the average reward of the current batch. The gradient of the loss function is formulated as

$$\nabla_\theta J(\theta) \approx \frac{1}{M} \sum_{m=1}^{M} [R(Y_m, Y^*) - b] \nabla_\theta \log \pi_\theta(Y_m|X), \quad (3)$$

where M is the number of sampled sequences per input. To maintain parameter efficiency and prevent catastrophic forgetting of the general structural knowledge, we employ LoRA [25] during this stage. This allows SALE to adapt to domain-specific DEE tasks while keeping the majority of the pre-trained Code-LLM parameters frozen, ensuring a lightweight footprint suitable for deployment.

4. Experiment

In this section, we conduct a comprehensive evaluation of the proposed **SALE** framework to assess its effectiveness and efficiency. Rather than isolating code pre-training as a single causal factor, our experiments are designed to evaluate the proposed paradigm through two primary objectives: first, whether our comprehensive framework—integrating a code-oriented backbone, Python-style structured prompting, and two-stage adaptation—effectively bridges the gap between general LLMs and the rigorous schema requirements of Document-level Event Extraction; and second, whether **SALE** can achieve state-of-the-art performance in low-resource settings while maintaining a lightweight parameter footprint suitable for practical deployment.

4.1. Datasets

To align with our two-stage training paradigm, we employ a tiered dataset strategy designed to verify both structural generalization and task-specific performance.

For the Structure-Aware Fine-tuning stage, we aggregate over 95,000 instruction-tuning instances from widely recognized IE benchmarks to inject general structural inductive bias. Specifically, we utilize ACE05 [28], a multilingual benchmark containing complex entity and event annotations; CoNLL-2003 [29], a standard dataset for Named Entity Recognition focusing on news corpora; and BC5CDR [30], a biomedical dataset for chemical–disease relation extraction which ensures domain diversity. The inclusion of heterogeneous datasets from diverse domains, such as the biomedical data in BC5CDR, provides the model with an initial capacity for structural transferability, laying a foundation for generalized Information Extraction (IE) prior to task-specific adaptation. As detailed in the Methodology, casting these heterogeneous tasks into Python-style definitions provides the model with a generalized understanding of structure.

For the Event Extraction Alignment stage, we evaluate the final DEE performance on two challenging benchmarks. We employ RAMS [31], a dataset specifically designed for multi-sentence argument linking with 9124 event instances where arguments frequently span across different sentences from the trigger. Complementing this, we use WikiEvents [8], derived from Wikipedia, which features complex event structures and coref-

erence chains to rigorously test the model's capacity for document-level understanding and argument classification.

Clarification on the Evaluation Setting. It is important to clarify our terminology regarding the evaluation setup. Following recent paradigms in evaluating LLMs for information extraction, the terms "zero-shot" and "few-shot" in this paper specifically refer to cross-task zero-shot transfer and cross-task few-shot transfer, respectively. While our model is instruction-tuned on general information extraction instances during Stage 1 (Structure Injection), it has never been exposed to the specific schemas, annotations, or training splits of the target DEE datasets (i.e., RAMS and WikiEvents) prior to evaluation. Therefore, our evaluation strictly measures the model's generalization and structural transferability to unseen tasks without any dataset-specific supervised fine-tuning. Furthermore, we clarify that while SALE leverages cross-domain knowledge from Stage 1, the current end-task evaluation on RAMS and WikiEvents is intended to validate the framework's effectiveness in news and general domains rather than serving as a exhaustive verification of robustness in specialized legal or multilingual extraction scenarios, which are deferred to future research.

4.2. Evaluation Metrics

Following standard protocols in Document-level Event Extraction [8,9], we evaluate performance using Argument Identification (Arg-I) and Argument Classification (Arg-C) F1 scores. Arg-I measures the model's ability to locate event participants; an argument is considered correctly identified if its predicted span offsets match the gold standard exactly. Arg-C measures the model's ability to understand the specific role of the participant within the event structure; an argument is considered correctly classified only if both its span offsets and its semantic role type (e.g., Buyer, Seller) match the gold standard exactly. We report the micro-averaged F1 scores for both metrics to provide a comprehensive assessment of the model's precision and recall.

4.3. Baseline Models

To evaluate the effectiveness of our proposed work under the current experimental setting, we conduct a comparative analysis against a range of baselines, categorized into task-specific supervised models and general-purpose LLMs.

Discussion on Comparison Fairness. It is important to acknowledge the distinct training paradigms between our approach and traditional baselines. While SALE benefits from a broad, multi-dataset Information Extraction (IE) instruction-tuning phase (Stage 1), the traditional supervised baselines are trained exclusively on the target DEE datasets (e.g., RAMS or WikiEvents). Therefore, comparing our cross-task transfer performance against their fully supervised performance is not intended as a strictly uniform data comparison. Rather, it serves to demonstrate that the generalized structural reasoning capabilities acquired by our lightweight LLM can match or even exceed the performance of models specifically optimized for the target domain.

Specific Supervised Models. We select three representative supervised models that employ distinct paradigms for event extraction:

- DyGIE++ [32]: A robust graph-based multi-task framework that models spans as nodes in a dynamic graph. It captures interactions between spans through graph propagation, effectively modeling dependencies for Entity Recognition, Relation Extraction, and Event Argument Extraction.
- RCEE [19]: An approach that reformulates Event Argument Extraction (EAE) as a Machine Reading Comprehension (MRC) task. By generating questions for each argument role, it extracts arguments from the context, demonstrating strong performance in capturing complex event relationships.

- EEQA [20]: A question-answering-based framework that converts event extraction into a series of natural language questions. It leverages the reading comprehension capabilities of pre-trained models (e.g., BERT) to extract event arguments, proving particularly effective for data-scarce scenarios.

Large Language Models. To assess the capabilities of generative models in the DEE task, we evaluate both closed-source and open-source LLMs under cross-task zero-shot and few-shot transfer settings:

- GPT-3.5-turbo: A powerful, proprietary general-purpose LLM accessed via OpenAI's API. We evaluate it to benchmark the performance of state-of-the-art foundation models that rely on in-context learning without task-specific parameter updates.
- CodeLLaMa-7B & Qwen2-7B: Representative open-source LLMs. CodeLLaMa is optimized for code generation, possessing strong structural reasoning abilities, while Qwen2 represents general text-generation capabilities. We evaluate their "vanilla" (general instruction-tuned but not DEE-specific) versions to establish a baseline for general-purpose models prior to our proposed structural adaptation.

We clarify that while our current baseline set includes representative supervised methods and general-purpose LLMs, it is not an exhaustive survey of all recent instruction-tuned structured extraction models or fine-tuned LLM-based IE frameworks. This selection is specifically intended to benchmark SALE against the two primary established paradigms—rigid supervision and flexible zero-shot generation—to evaluate its capacity to bridge the gap between them. Incorporating newer, specialized instruction-tuned baselines remains an important direction for future evaluation.

4.4. Implementation Details

To ensure reproducibility and computational efficiency, all experiments were conducted on a high-performance computing cluster running the Linux Ubuntu 24.04 operating system. The hardware infrastructure is equipped with Intel Xeon Platinum 8468V CPUs and NVIDIA A40 GPUs (40 GB VRAM). The software environment is containerized using Docker to maintain consistency, integrating PyTorch 1.9, the HuggingFace and Transformers library, and the PEFT library for efficient fine-tuning.

For the backbone of our framework, we primarily utilize CodeLLaMa-7B [33] due to its superior performance in handling structured data compared to general text-generation models. We adopt a progressive training strategy:

- Stage 1 (Structure-Aware Fine-Tuning): The model was fine-tuned on the unified IE dataset for approximately 120 h. We utilized an effective batch size of 32 and a learning rate of 3×10^{-4} to ensure stable convergence on the diverse multi-task corpus.
- Stage 2 (Event Extraction Alignment): To mitigate the computational overhead while adapting to the specific DEE tasks, we employed Low-Rank Adaptation (LoRA). This parameter-efficient fine-tuning approach allows us to optimize the specific Argument Identification (Arg-I) and Argument Classification (Arg-C) objectives without updating the full model parameters.

4.5. Main Results

Table 4 presents a comprehensive performance comparison of our proposed method against various baselines on the RAMS and WikiEvents datasets.

Table 4. Comparison of argument extraction performance (F1 Score). * indicates statistically significant improvements over the best baseline ($p < 0.05$, paired t-test).

Model	Setting	RAMS		WikiEvents(EAE)	
		Arg-I	**Arg-C**	**Arg-I**	**Arg-C**
Supervised Models					
DyGIE++ [32]	Supervised	44.4	35.3	39.8	35.3
RCEE [19]	Supervised	45.4	41.5	53.7	50.9
EEQA [20]	Supervised	48.9	44.7	54.3	**51.7**
LLMs (Cross-task Transfer)					
Qwen2-7B [34]	0-shot	4.2	17.2	10.3	14.3
Qwen2-7B [34]	2-shot	20.1	20.0	-	-
CodeLLaMa-7B [33]	0-shot	7.5	18.4	10.0	15.2
CodeLLaMa-7B [33]	2-shot	21.5	20.4	-	-
Ours (CodeLLaMa+SFT)	0-shot	52.6 *	43.9 *	50.2	27.0
Ours (CodeLLaMa+SFT)	2-shot	**54.6** *	**49.4** *	**54.6** *	30.0

As indicated in Table 4, standard LLMs such as CodeLLaMa-7B and Qwen2-7B exhibit suboptimal performance in the cross-task zero-shot transfer setting, with Arg-I scores falling below 10% on the RAMS dataset. This deficiency highlights the significant challenge general-purpose models face in grasping the intricate schema constraints and context-dependent nature of Document-level Event Extraction (DEE) without our proposed structural adaptation. While providing few-shot examples (2-shot) yields some improvement, the performance remains well below that of specialized supervised models. In contrast, our proposed Structure-Aware Fine-tuned (SFT) model demonstrates substantial performance gains. On the RAMS dataset, our method in the cross-task zero-shot setting significantly outperforms the strong supervised baseline RCEE by 7.2% in Arg-I (52.6% vs. 45.4%) and achieves competitive results against the state-of-the-art EEQA model. Furthermore, in the few-shot transfer setting (2-shot), our model attains the highest performance with an Arg-I of 54.6% and an Arg-C of 49.4%. To ensure the robustness of our findings, we conducted a paired t-test, confirming that the performance improvements of SALE over the baselines on RAMS are statistically significant ($p < 0.05$).

These results help clarify that the observed improvements cannot be attributed to instruction-data scale alone. Under the same unified IE corpus used for Stage 1 fine-tuning (Section 3.2.3), vanilla CodeLLaMa and Qwen still struggle to capture document-level arguments reliably. In contrast, SALE achieves a substantial performance gain, suggesting that the improvement arises from the combined effect of the code-oriented backbone, the Python-style structured prompting, and the proposed two-stage adaptation strategy, which together encourage stronger schema adherence and structured reasoning. We therefore interpret the gains as being primarily associated with the structural prompting and alignment framework enabled by the code-form representation, rather than being explained solely by the amount of instruction data.

The performance difference between CodeLLaMa-7B and LLaMa2-7B in our experiments is relatively small, so the current results should not be interpreted as strong evidence that code-based pretraining alone provides a substantial structural inductive bias; rather, the observed gains likely reflect the combined effect of the code-oriented backbone, schema-guided structured prompting, and the proposed two-stage training strategy.

Crucially, our qualitative error analysis reveals that these substantial F1 score improvements are intrinsically linked to a marked reduction in structural hallucinations. While

vanilla LLMs frequently generate outputs that violate the prescribed event schema—often resulting in unparsable formats or hallucinated argument roles—the combination of SALE's code-oriented backbone, structured prompting, and two-stage adaptation effectively constrains the generative space. By framing the output strictly as a Python class instantiation, the proposed paradigm minimizes formatting errors. We note that while these observed gains demonstrate the effectiveness of the overall code-form structured prompting and adaptation paradigm, we do not assert that code pre-training alone acts as the single causal factor isolated under fully controlled conditions. Instead, this qualitative evidence suggests that maintaining the structural validity of the generation is a key driver in conquering complex DEE tasks.

We clarify that while our current baseline set covers representative supervised methods and general-purpose LLMs, it is not an exhaustive survey of all recent instruction-tuned structured extraction models or fine-tuned LLM-based IE frameworks. The comparative results are intended to demonstrate SALE's capacity to bridge the gap between established paradigms rather than provide a comprehensive ranking against all emerging IE systems. Incorporating newer specialized instruction-tuned baselines is acknowledged as an essential direction for future evaluation.

4.6. Ablation Study

To rigorously evaluate the efficacy of our proposed training strategy and the model's capacity for cross-sentence reasoning and structural validity, we conduct an ablation study and a qualitative case analysis.

Impact of Training Stages. We examine the incremental contribution of the Structure-Aware Fine-tuning (Stage 1) and the subsequent Event Extraction Alignment (Stage 2). We define "BaseModel" as the model fine-tuned solely on general Information Extraction (IE) datasets (Stage 1), and compare its performance against our final model, which undergoes both Stage 1 and Stage 2 training. All evaluations are conducted under the cross-task zero-shot or few-shot transfer settings.

The empirical results in Table 5 demonstrate that while the Structure-Aware Fine-tuning establishes a solid foundation (achieving ∼44% Arg-I), the Event Extraction Alignment is critical for domain-specific adaptation. The second stage confers a significant performance boost, elevating the Arg-I by 8.2% in the cross-task zero-shot setting. The effectiveness of these stages is fundamentally underpinned by the synergy between our Python-based prompt design and the alignment loss formulation. The prompt template activates the Code-LLM's structural reasoning capabilities, while the alignment weighting functions as a critical governor, enforcing cross-sentence consistency and schema adherence during the adaptation process.

Table 5. Ablation Study on RAMS Dataset (F1 Score).

Model	Setting	Arg-I	Arg-C	Improvement
BaseModel (Stage 1 only)	0-shot	44.4	35.3	-
Ours (Stage 1 + Stage 2)	**0-shot**	**52.6**	**43.9**	**+8.2/+8.6**
BaseModel (Stage 1 only)	2-shot	45.3	42.1	-
Ours (Stage 1 + Stage 2)	**2-shot**	**54.6**	**49.4**	**+9.3/+7.3**

Case Study on Cross-Sentence Reasoning and Structural Validity. To provide a deeper understanding of how the combination of our Python-style structured prompts and the code-oriented backbone operates in practice to minimize schema violations, we conduct a qualitative analysis based on the examples visualized in Figure 3. The figure depicts a com-

plex financial document describing a "TransferOwnership" scenario involving TechCorp and InnovateX, alongside a secondary "Divestiture" event related to OldHoldings.

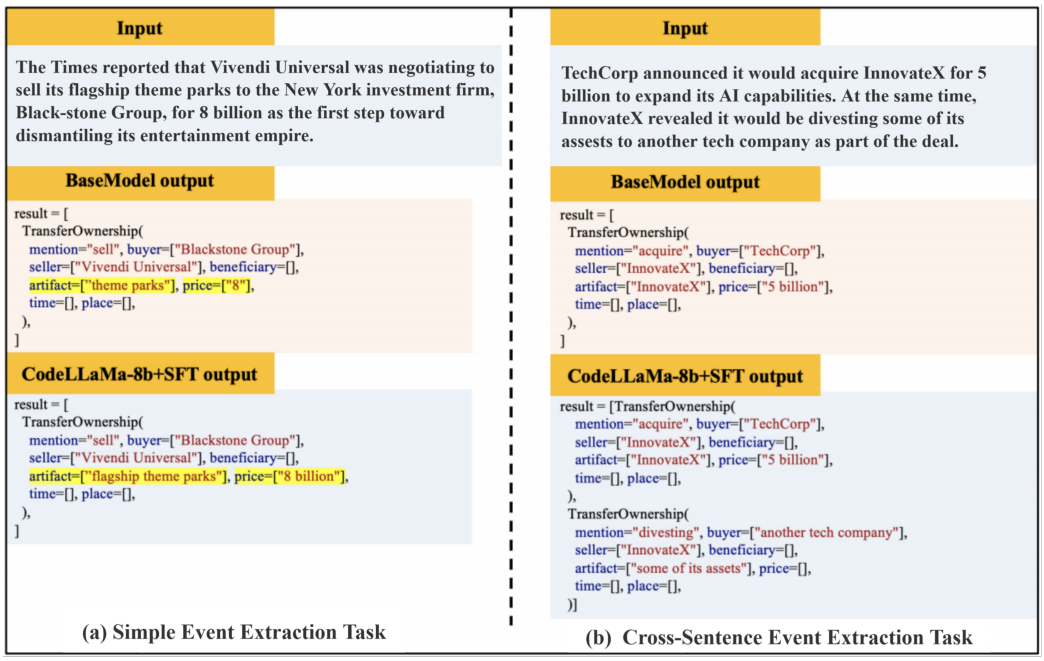

Figure 3. Qualitative comparison of cross-sentence argument extraction between the Baseline (General LLM) and SALE. The example illustrates a complex TransferOwnership scenario where event triggers and arguments (e.g., Seller and Artifact) are dispersed across multiple sentences. The Baseline model suffers from context loss, failing to link the distant arguments or hallucinating incorrect roles (leading to schema violations). In contrast, **SALE** successfully instantiates the target Python class with precise argument assignments. This demonstrates the effectiveness of our proposed paradigm, where the synergy between Python-style structured prompting and a code-oriented backbone effectively handles cross-sentence dependencies by maintaining "variable scope" within the class constructor context.

In the first case, which focuses on the primary acquisition event, the challenge lies in the significant distance between the event trigger "acquired" and the argument InnovateX, which appears three sentences later. The baseline Qwen2-7B model fails to capture this long-range dependency, incorrectly predicting a local entity as the object or omitting it entirely. This failure highlights the tendency of general LLMs to prioritize local semantic consistency over global structural integrity. In contrast, **SALE** successfully links InnovateX to the Artifact role. Rather than attributing this success solely to an inherent code-centric inductive bias, we observe that it is the integration of the code-oriented backbone with our rigorous Python-class prompting that allows the model to maintain "variable scope" across extensive text. By treating the document as a code block, SALE effectively extends the attention span of the event constructor to encompass distant arguments, resolving the cross-sentence dependency that baffles general text models.

The second case illustrates a multi-event coexistence scenario involving the divestiture of assets by OldHoldings. Here, the baseline model exhibits "role confusion," conflating the arguments of the acquisition event with those of the divestiture event, resulting in a merged and structurally invalid output. Conversely, SALE correctly instantiates a separate Divestiture class instance, keeping its arguments distinct from the TransferOwnership event. This precise separation demonstrates that our structured prompts and alignment framework, built on a code-oriented backbone, provide a highly effective representation for multi-event extraction. This success is a direct result of the reward-weighted alignment and our class-based prompt structure, which specifically penalize the 'role confusion' and argument leakage common in standard LLMs. By combining object-oriented prompting

with task-specific alignment weighting, SALE ensures that each event's argument 'scope' is strictly maintained, even when multiple events coexist in a single document. While we do not report a separate schema violation metric due to the pipeline's end-to-end evaluation nature, this qualitative evidence directly addresses the issue of structural validity. Just as distinct objects in programming encapsulate their own states without interference, SALE treats concurrent events as independent class instances, thereby naturally mitigating the issue of argument overlap and hallucination that plagues standard generative approaches.

Analysis on Computational Efficiency. To justify the "Lightweight" claim of our framework, we conduct a comparative analysis of computational resource requirements between SALE and other LLM-based paradigms. We define efficiency from two critical dimensions: Parameter Efficiency (ratio of trainable parameters) and Deployment Feasibility (hardware constraints). Table 6 summarizes the resource demands. Compared to giant general models like Qwen2-72B or GPT-4 (accessed via API with high latency and privacy risks), SALE employs a compact 7B backbone. More importantly, unlike standard Full Fine-Tuning (FFT) which requires updating 100% of parameters and demands expensive A100 clusters (often exceeding 80 GB VRAM due to optimizer states), our Stage 2 utilizes Low-Rank Adaptation (LoRA). SALE requires updating only approximately 0.06% of the parameters. This drastic reduction allows the model to be trained on a single consumer-grade GPU (e.g., RTX 3090/4090 with 24 GB VRAM), requiring only ∼16 GB VRAM. This demonstrates that SALE achieves state-of-the-art performance with a minimal footprint, making it a practical solution for vertical domains where computing resources are limited and data privacy is paramount.

Beyond parameter efficiency, we further evaluate the deployment feasibility of SALE through the lens of inference performance and scalability. Regarding inference latency and throughput, since SALE utilizes Low-Rank Adaptation (LoRA) for task alignment, the optimized weights can be merged into the frozen backbone (CodeLLaMA-7B) without altering the transformer architecture or adding extra computational modules. Consequently, SALE maintains an inference profile identical to the base CodeLLaMA-7B model, avoiding the additional latency often introduced by complex graph-based or multi-decoder architectures. Furthermore, our framework exhibits high scalability as the structure-aware prompting and alignment paradigm are model-agnostic. This allows SALE to be seamlessly transitioned to larger or more advanced code-centric backbones, ensuring its practical utility as foundation models continue to evolve.

Table 6. Comparison of Computational Efficiency and Deployment Requirements. SALE achieves an optimal balance between performance and resource consumption. By leveraging LoRA on a 7B Code-LLM backbone, it significantly lowers the entry barrier for deployment compared to Full Fine-Tuning (FFT) or Giant LLMs.

Model Paradigm	Backbone Scale	Tuning Method	Trainable Params	Min. VRAM (Train)	Hardware Entry
Giant General LLMs					
GPT-4	≈1.8T (Est.)	In-Context Learning	0	N/A (API)	Cloud API
Qwen2-72B	72B	Full Fine-Tuning	100%	>144 GB	4×A100 (80 G)
7B General LLMs					
Llama-2-7B	7B	Full Fine-Tuning	100%	≈28 GB	2×RTX 3090
CodeLLaMa-7B (Base)	7B	Full Fine-Tuning	100%	≈28 GB	2×RTX 3090
Ours					
SALE	**7B**	**LoRA (PEFT)**	**∼0.06%**	**≈16 GB**	**1×RTX 3090**

5. Conclusions

In this work, we propose a novel Structure-Aware Lightweight DEE (SALE) framework to address the inductive bias mismatch in Document-level Event Extraction. The core of our

methodology is the strategic utilization of Code-LLMs as a favorable inductive preference via a Python class instantiation paradigm. Thanks to this code-centric design coupled with our two-stage structural adaptation strategy, our model is able to bridge natural language semantics with rigid structural constraints, effectively handling cross-sentence dependencies. Our findings indicate that the superior performance of SALE is not solely a consequence of code-based pre-training, but rather reflects the synergistic effect of the code-oriented backbone, our schema-aware structured prompting, and the proposed two-stage alignment pipeline. This ensures the structural validity of the generated outputs and mitigates the structural hallucinations common in general LLMs, while achieving highly efficient extraction without relying on massive parameters.

Evaluations on established benchmarks (RAMS and WikiEvents) demonstrate that SALE achieves superior performance in cross-task zero-shot and few-shot transfer scenarios within the evaluated news and general domains. While these results confirm the structural reasoning potential of Code-LLMs, we acknowledge that the current comparative analysis primarily benchmarks SALE against representative supervised and general-purpose LLM baselines. Future research will focus on expanding this evaluation to include a wider array of recent instruction-tuned structured extraction models, alongside further exploration into specialized domains (e.g., legal or clinical) and multilingual settings for future evaluation to further establish the framework's robustness across diverse real-world scenarios. We hope this work could bring the community new insights for advanced code-driven structured prediction pipelines and lightweight domain-specific information extraction.

Author Contributions: Conceptualization, X.X., J.Z., P.Z. (Pengfei Zhang), Y.L. and B.Y.; methodology, P.Z. (Pengfei Zhang), D.H. and Z.D.; validation, P.Z. (Puyuan Zheng), D.H. and Z.D.; writing—original draft preparation, X.X., Y.L., B.Y., P.Z. (Puyuan Zheng), D.H., Z.D., P.Z. (Ping Zong), G.Z. and Y.Z.; writing—review and editing, X.X., P.Z. (Ping Zong), G.Z., Z.O., M.S. and Y.Z.; supervision, P.Z. (Ping Zong), G.Z., Z.O., M.S. and Y.Z.; project administration, J.Z., Z.O. and M.S.; funding acquisition, J.Z. and P.Z. (Pengfei Zhang). All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Science and Technology Project of State Grid Hebei Information and Telecommunication Branch grant number kj2024-018. The APC was funded by the Science and Technology Project of State Grid Hebei Information and Telecommunication Branch.

Data Availability Statement: There are no restrictions on the sharing of relevant data in this study.

Acknowledgments: During the preparation of this manuscript, the authors used Gemini 3 solely for language refinement, including grammar, phrasing, and text editing. The AI tool did not participate in the research design, data analysis, experimental development, scientific interpretation, or the generation of technical content. All AI-assisted text was fully reviewed, verified, and revised by the authors, who take full responsibility for the final scientific content. All authors have agreed to this acknowledgment.

Conflicts of Interest: Authors Xing Xu, Jianbin Zhao, Pengfei Zhang, Yaduo Liu and Bingyang Yu were employed by the company State Grid Hebei Information and Telecommunication Branch. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Ye, J.; Chen, X.; Xu, N.; Zu, C.; Shao, Z.; Liu, S.; Cui, Y.; Zhou, Z.; Gong, C.; Shen, Y.; et al. A comprehensive capability analysis of GPT-3 and GPT-3.5 series models. *arXiv* **2023**, arXiv:2303.10420. [CrossRef]
2. Chen, X.; Ye, J.; Zu, C.; Xu, N.; Zheng, R.; Peng, M.; Zhou, J.; Gui, T.; Zhang, Q.; Huang, X. How robust is GPT-3.5 to predecessors? A comprehensive study on language understanding tasks. *arXiv* **2023**, arXiv:2303.00293. [CrossRef]
3. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **2017**, *30*, 5998–6008.

4. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **2020**, *33*, 1877–1901.
5. Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. Training language models to follow instructions with human feedback. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **2022**, *35*, 27730–27744.
6. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. GPT-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [CrossRef]
7. Raihan, N.; Newman, C.; Zampieri, M. Code llms: A taxonomy-based survey. In Proceedings of the IEEE International Conference on Big Data (BigData), Washington, DC, USA, 15–18 December 2024; pp. 5402–5411.
8. Li, S.; Ji, H.; Han, J. Document-Level Event Argument Extraction by Conditional Generation. In *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 894–908.
9. Peng, J.; Sun, H.; Yang, W.; Wei, F.; He, L.; Wang, L. One small and one large for document-level event argument extraction. *arXiv* **2024**, arXiv:2411.05895. [CrossRef]
10. Yu, H.; Wang, K.; Cao, H.; Cui, Y. Document-level event argument extraction with dual-paradigm fusion strategy. *J. King Saud Univ. Comput. Inf. Sci.* **2025**, *37*, 322. [CrossRef]
11. Zhou, J.; Shuang, K.; Wang, Q.; Yao, X. Eace: A document-level event argument extraction model with argument constraint enhancement. *Inf. Process. Manag.* **2024**, *61*, 103559. [CrossRef]
12. Zhang, M.; Chen, H. Document-Level Event Argument Extraction with Sparse Representation Attention. *Mathematics* **2024**, *12*, 2636. [CrossRef]
13. Min, Q.; Qu, Z.; Guo, Q.; Hu, X.; Zhang, Z.; Zhang, Y. Multi-Document Event Extraction Using Large and Small Language Models. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 19265–19296.
14. Wang, Y.; Yu, B.; Liu, Y.; Lu, S. True-uie: Two universal relations unify information extraction tasks. In *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers) (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 1863–1876.
15. Wang, X.; Zhou, W.; Zu, C.; Xia, H.; Chen, T.; Zhang, Y.; Zheng, R.; Ye, J.; Zhang, Q.; Gui, T.; et al. Instructuie: Multi-task instruction tuning for unified information extraction. *arXiv* **2023**, arXiv:2304.08085. [CrossRef]
16. Jiao, Y.; Zhong, M.; Li, S.; Zhao, R.; Ouyang, S.; Ji, H.; Han, J. Instruct and Extract: Instruction Tuning for On-Demand Information Extraction. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 10030–10051.
17. Hsu, I.H.; Huang, K.H.; Boschee, E.; Miller, S.; Natarajan, P.; Chang, K.W.; Peng, N. DEGREE: A data-efficient generation-based event extraction model. In *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 1890–1908.
18. Lin, R.; Liu, Y.; Gan, Y.; Cai, Y.; Lan, T.; Liu, Q. GEMS: Generation-Based Event Argument Extraction via Multi-perspective Prompts and Ontology Steering. In *Proceedings of the Association for Computational Linguistics: ACL*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2025; pp. 26392–26409.
19. Liu, J.; Chen, Y.; Liu, K.; Bi, W.; Liu, X. Event extraction as machine reading comprehension. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 1641–1651.
20. Du, X.; Cardie, C. Event Extraction by Answering (Almost) Natural Questions. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 671–683.
21. Lu, Y.; Liu, Q.; Dai, D.; Xiao, X.; Lin, H.; Han, X.; Sun, L.; Wu, H. Unified Structure Generation for Universal Information Extraction. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 5755–5772.
22. Fei, H.; Wu, S.; Li, J.; Li, B.; Li, F.; Qin, L.; Zhang, M.; Zhang, M.; Chua, T.S. Lasuie: Unifying information extraction with latent adaptive structure-aware generative language model. *Adv. Neural Inf. Process. Syst. (NeurIPS)* **2022**, *35*, 15460–15475.
23. Li, X.L.; Liang, P. Prefix-Tuning: Optimizing Continuous Prompts for Generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers) (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 4582–4597.
24. Liu, X.; Ji, K.; Fu, Y.; Tam, W.; Du, Z.; Yang, Z.; Tang, J. P-tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers) (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 61–68.
25. Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; De Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; Gelly, S. Parameter-efficient transfer learning for NLP. In Proceedings of the 36th International Conference on Machine Learning (ICML), PMLR, Long Beach, CA, USA, 10–15 June 2019; pp. 2790–2799.

26. Lester, B.; Al-Rfou, R.; Constant, N. The Power of Scale for Parameter-Efficient Prompt Tuning. In *Proceedings of the Empirical Methods in Natural Language Processing (EMNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2021; pp. 3045–3059.
27. Williams, R.J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.* **1992**, *8*, 229–256. [CrossRef]
28. Walker, C.; Strassel, S.; Medero, J.; Maeda, K. Ace 2005 multilingual training corpus. *Linguist. Data Consort.* **2006**. [CrossRef]
29. Sang, E.T.K.; De Meulder, F. Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL, Edmonton, AB, Canada, 31 May–1 June 2003; pp. 142–147.
30. Li, J.; Sun, Y.; Johnson, R.J.; Sciaky, D.; Wei, C.H.; Leaman, R.; Davis, A.P.; Mattingly, C.J.; Wiegers, T.C.; Lu, Z. BioCreative V CDR task corpus: A resource for chemical disease relation extraction. *Database* **2016**, *2016*, baw068. [CrossRef] [PubMed]
31. Ebner, S.; Xia, P.; Culkin, R.; Rawlins, K.; Van Durme, B. Multi-sentence argument linking. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 8057–8077.
32. Wadden, D.; Wennberg, U.; Luan, Y.; Hajishirzi, H. Entity, Relation, and Event Extraction with Contextualized Span Representations. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019.
33. Roziere, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code Llama: Open foundation models for code. *arXiv* **2023**, arXiv:2308.12950. [CrossRef]
34. Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; Li, C.; Liu, D.; Huang, F.; et al. Qwen2 technical report. *arXiv* **2024**, arXiv:2407.1067. [CrossRef]